

Tutorial Sheet 3

Files

AUTHOR
Operating Systems Teaching and Research Unit

PUBLISHED
02.06.2024

Introduction

Question 1

Some operating systems provide a system call `rename` to give a file a new name. Is there any difference at all between using this call to rename a file and just copying the file to a new file with the new name, followed by deleting the old one?

Question 2

If `/home/root/` is the current working directory, what is the absolute path name for the file whose relative path name is `../redha/BuS/sched.txt` ?

Question 3

Contiguous allocation of files leads to disk fragmentation, because some space in the last disk block will be wasted in files whose length is not an integral number of blocks. Describe the internal and external fragmentation during the allocation and deletion of block.s

Question 4

A UNIX file system has 4 KiB blocks and 4-bytes disk addresses. What is the maximum file size in bytes if the inode contains 10 direct entries, and one single, double and triple indirect entry each?

Directory entries

`/bin`

inode	entry
100282	automake
2189328	base64
2189329	basename
938649	bash
260030	bc
3481110	btrfs
1701644	bzip2
1701645	bzip2recover
2244837	chpasswd
2189336	chroot
938819	sh

`/usr/bin`

inode	entry
3855844	curl
3627359	ip
1698762	pkill
2189417	tty
3375120	virsh
3807607	xetex
2242395	xz
2242396	xzcat
938819	sh

`/usr`

inode	entry
626	bin
627	include
628	lib
59563	lib32
2381503	lib64
3855781	libexec
640	local
267721	resources
2381505	sbin
581	share
686	src

Question 5

Can you spot the hard links in these directory entries? What would the file `/bin/sh` contain if it was a symbolic link to `bash`?

Question 6

What would `/usr/lib64` contain if it was a symbolic link to the directory `/usr/lib`? Is this possible with a hard link?

Question 7

Name one advantage of symbolic link over hard links and one advantage of hard links over symbolic links.

Question 8

How many disk operations are needed to fetch the inode for a file with the path name `/usr/ast/courses/os/handout.t`? Assume that the inode for the root directory is in memory, but nothing else along the path is in memory. Also assume that all directories fit in one disk block.

How can we reduce that number of disk operations?

Question 1

Basic file system (like FAT and EXT seen during the lecture) copy the file block by block for the syscall `copy`, and they create a new inode associated with the new copied file. On the other hand, renaming a file is just updating the name of the file inside the directory, the only IO happens when getting the directory block to update the file name. This is much more efficient, especially when dealing with big files.

Question 2

The full path name is `/home/redha/BuS/sched.txt` because the special path character `..` goes one directory behind, relative to `/home/root/`

Question 3

We have internal fragmentation because there is wasted space inside allocated blocks, if the file is not exactly a multiple of the block size. The moment we start to delete files, we will be deleting blocks and create external fragmentation as well.

Question 4

A block can contain $4 \text{ KiB} / 4 \text{ B} = 1024$ pointers.

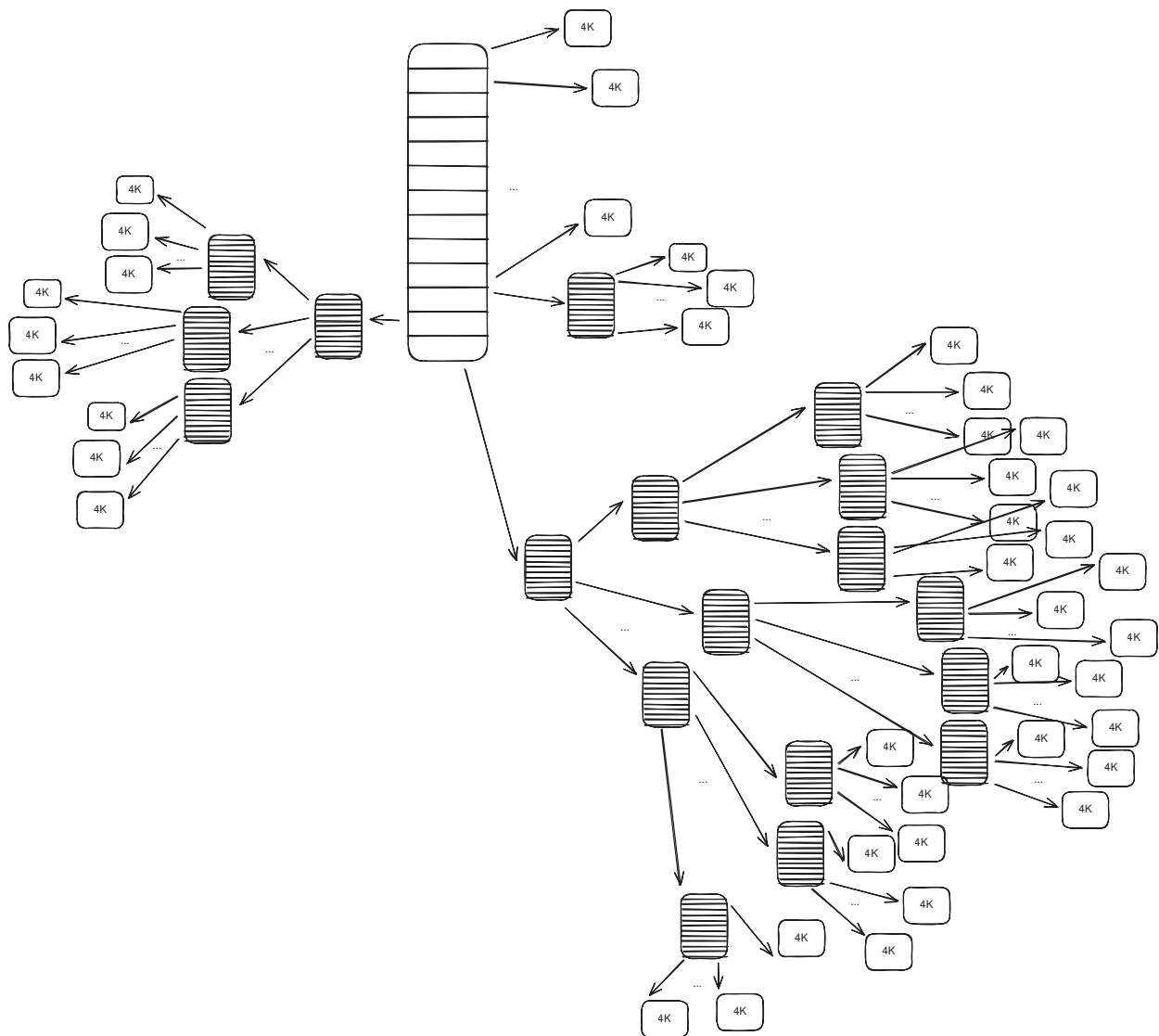
The maximum size is:

- $10 * 4 \text{ KiB}$ for the direct blocks
- $1024 * 4 \text{ KiB}$ for the single indirect block
- $1024^2 * 4 \text{ KiB}$ for the double indirect block
- $1024^3 * 4 \text{ KiB}$ for the triple indirect block

For a total of 4402345713664 bytes (around 4 TB)

Question 5

Maximum file size



`/bin/sh` and `/usr/bin/sh` contain the same inode number. They both point to the same file therefore they are hard links. If `/bin/sh` is a symbolic link to `bash`, then the file associated with `/bin/sh` (inode 938819) contains a path to `/bin/bash`.

Question 6

If `/usr/lib64` is a symbolic link to `/usr/lib`, then the entry is also a file containing a path to `/usr/lib`. This is not possible with a hard link

Question 7

Symbolic link allow link between folders and across partitions, that is not possible for hard links. Hard link don't have the problem of broken links, if the file is deleted, and another hard links exists, the link will still be valid. With symbolic links, if the original file (inode) is deleted, all the other link will become "broken".

Question 8

- 1 disk IO for fetching the block directory for `/` (inside the block, we get the inode number of `usr`)
- 1 disk IO for fetching the inode of `/usr`
- 1 disk IO for fetching the block directory for `/usr` (we get the inode number of `ast`)

- 1 disk IO for fetching the inode of `/usr/ast`
- 1 disk IO for fetching the block directory for `/usr/ast`
- 1 disk IO for fetching the inode of `/usr/ast/courses`
- 1 disk IO for fetching the block directory for `/usr/ast/courses`
- 1 disk IO for fetching the inode of `/usr/ast/courses/os`
- 1 disk IO for fetching the block directory for `/usr/ast/courses/os`
- 1 disk IO for fetching the inode of `/usr/ast/courses/os/handout.t`

10 disk accesses in total. To reduce that number (and because disk IO is very expensive), modern operating systems usually have an associative cache between the directory entries and the inode number.

Basic file management

In this exercise, you will learn how to:

1. read the manual pages for system calls
2. access files at low level

1. Read the manual

Read and understand the `open` syscall on the manual, what are the arguments, what is the return value?

💡 Tip

On the Linux terminal, type `man 2 open`. If you don't have access to a Linux terminal, all man pages can be found on the Internet.

How will you call this function to create a new file? What does the flag `O_TRUNC` is doing? How to combine multiple flags?

Solution

The arguments are:

- a `string` that represent the pathname of the file
- a `int` that represent flags
- an optional argument `mode` used when creating a new file

On success, the function returns the new file descriptor, on error, -1 is returned.

Creating a new file can be made using the flag `O_CREAT`, and we need to add the mode (refer to UNIX permissions on the manual of `open()`).

```
1 int fd = open("new_file", O_CREAT, 00700);
```

The flag `O_TRUNC` is used to truncate a file upon open, which discards all data inside the file.

Finally, to combine multiple flags, we use the OR operator. For example, to write to a file and discard the current data:

```
1 int fd = open("file", O_RDWR | O_TRUNC);
```

2. Open and modify a file

Write a C code that opens a file, that modifies all the occurrences of the char `'a'` to `'b'`, starting at the offset 592, until the end of file.

Solution

```
1  int main() {
2      int fd = open("file", O_RDWR);
3      if (fd < 0) {
4          perror("open");
5          exit(1);
6      }
7
8      int ret = lseek(fd, 592, SEEK_SET);
9      if (ret == -1) {
10         perror("lseek");
11         goto out;
12     }
13
14     char buf;
15
16     do {
17         ret = read(fd, &buf, 1);
18         if (ret != 1) {
19             goto out;
20         }
21         if (buf == 'a') {
22             buf = 'b';
23             if (lseek(fd, -1, SEEK_CUR) == -1) {
24                 perror("lseek\n");
25                 exit(1);
26             }
27             ret = write(fd, &buf, 1);
28         }
29     } while(ret == 1);
30
31 out:
32     close(fd);
33 }
```

Understanding relations between file descriptor, file table and inodes

In this task, we'll be analyzing the state of file descriptors, file structures, and inodes in a parent and child process at specific points in the program execution.

```

1  int main()
2  {
3      char buffer[16];
4
5      int fd0 = open("file1.txt", O_RDONLY);
6      int fd1 = open("file2.txt", O_RDWR);
7      int fd2 = open("file3.txt", O_RDONLY, O_TRUNC);
8      int fd3 = dup(fd2);
9
10     lseek(fd0, 19, SEEK_SET);
11     lseek(fd1, 122, SEEK_SET);
12     lseek(fd2, 5, SEEK_SET);
13     read(fd3, buffer, 16);
14
15     if (fork() == 0) {
16         close(fd0);
17         close(fd2);
18
19         int fd4 = open("file5.txt", O_RDONLY);
20
21         lseek(fd4, 12, SEEK_SET);
22         write(fd1, BUFFER, 16);
23         lseek(fd3, -21, SEEK_CUR);
24
25         close(fd1);
26         close(fd3);
27         close(fd4);
28         return;
29     }
30
31     close(fd0);
32     close(fd1);
33     close(fd2);
34     close(fd3);
35 }

```

💡 Tip

Check the man page for `dup` to understand line 8.

Please note that `file1.txt` and `file2.txt` are hard links. Suppose that no file descriptor is in use at the start of the program, except for the standard input, standard output and standard error.

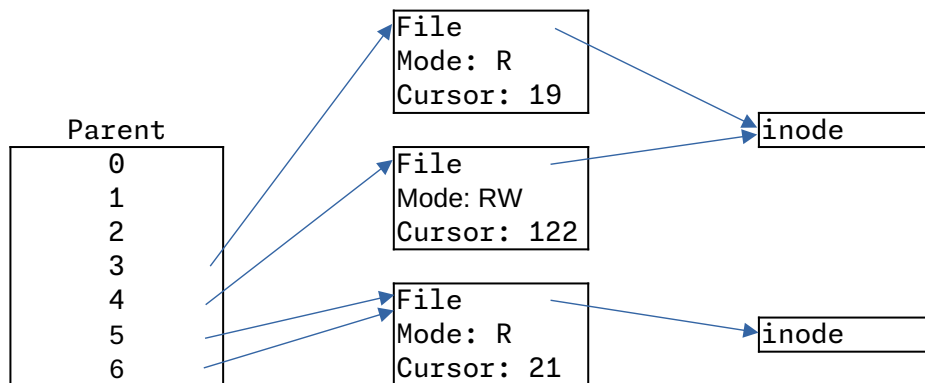
Task

At the specified lines (14 and 24), draw the state of:

- The file descriptor table for each process.
- The associated file structures (including inode, mode, and cursor).
- The corresponding inode.

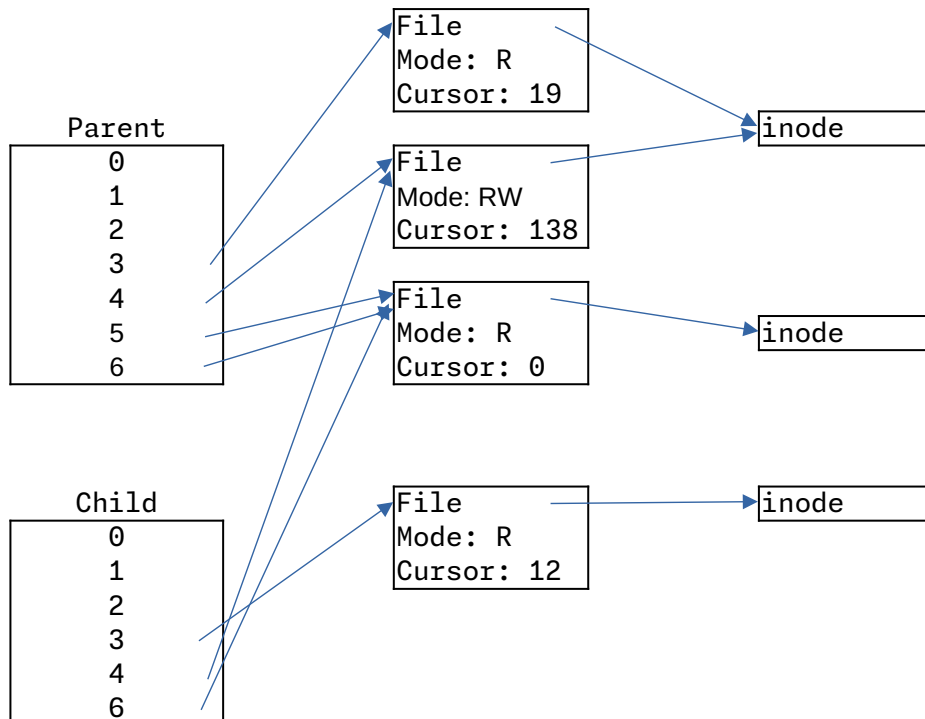
i Solution

Diagrams at line 14



- file descriptor 0, 1, 2 correspond respectively to stdin, stdout and stderr
- there is 1 structure file per call to `open()`
- `file1.txt` and `file2.txt` are hard links so they share the same inode
- `dup(fd2)` duplicated the file descriptor 5 to the file descriptor 6
- the cursors (or offset) are updated for each call of `lseek`, `read` and `write`

Diagrams at line 24



- the child inherits all the file descriptors of its parent
- in our code, the child closes the files associated with the files descriptor 3 and 5
- when it opens `file5.txt`, the first file descriptor that is not in use is 3, it now points to the new file

Questions

1. If the file permission of `file1.txt` was `0444`, what would be the result of executing the previous code?

Solution

1. `444` is, in UNIX permission: `r--r--r--` (read for others, groups and owners). Line 6, the call to `open` would fail and return `-1` because we cannot open a file with write permission if the file is read-only. (and we should check for this error in the code!).

Coding exercises

Dup 2 manipulation

Tip

In this task, you'll need `dup2` to do redirection. Check the man page, how is it different from `dup`?

Write a C function that executes a `command` with the arguments `args`. The function should call `fork()`, then executes the program with `execv()` and redirect the output of the command to the file `output_file`.

```
1 int stdout_to_file(char *command, char *args[], char *output_file)
2 {
3     ...
4 }
```

Solution

`dup2()` will replace a file descriptor by another one. In our case, we want to replace the file descriptor of the standard output with the file descriptor of the file `output_file`.

If the file descriptor of the `output_file` is `fd`, we can use: `dup2(fd, 1)`.

If we want to add the `stderr` as well, we can add to the code a call to `dup2(fd, 2)` where 2 is the file descriptor for the `stderr` by default.

```
1  int stdout_to_file(char *command, char *args[], char *output_file)
2  {
3      // ensure the file is created and truncated when it's opened
4      int fd = open(output_file, O_RDWR | O_CREAT | O_TRUNC, 00777);
5      pid_t pid;
6      int wstatus;
7      if (fd < 0) {
8          perror("open");
9          return -1;
10     }
11
12     pid = fork();
13     if (pid == 0) {
14         // the macro STDOUT_FILENO is equal to 1
15         if (dup2(fd, STDOUT_FILENO) == -1) {
16             perror("dup2");
17             exit(1);
18         }
19         // now, stdout is closed, instead the file descriptor is pointing to
20         // the output file
21         if (execv(command, args) == -1) {
22             perror("execv");
23             exit(1);
24         }
25     }
26     if (waitpid(pid, &wstatus, 0) != pid) {
27         perror("waitpid");
28         close(fd);
29         exit(1);
30     }
31     close(fd);
32     return wstatus;
33 }
```

Implement the UNIX cat command

Implement the following C function that concatenate all the files passed in argument. It should write the content of the files to the standard output.

```
1  int cat(char *filenames[], int nr_files)
2  {
3      ...
4  }
```

Solution

```
1  int cat(char *filenames[], int nr_files)
2  {
3      char *filename;
4      int fd;
5      char buffer[BUFFER_SIZE];
6      size_t size;
7
8      for (int i = 0; i < nr_files; i++) {
9          filename = filenames[i];
10         fd = open(filename, O_RDONLY);
11         if (fd < 0) {
12             perror("open");
13             continue;
14         }
15
16         size = BUFFER_SIZE;
17         while((size = read(fd, buffer, BUFFER_SIZE)) > 0) {
18             write(STDOUT_FILENO, buffer, size); // write to stdout (or we could use printf)
19         }
20
21         if (size < 0) {
22             perror("read");
23         }
24         close(fd);
25     }
26     return 0;
27 }
```